Hybrid Multi-Agent AI

Complete Master Reference Guide

Framework Deep-Dives | Step-by-Step Functionality | Comparative Analysis | Pricing

1. What is Agentic AI?

Agentic AI is an orchestration-based AI architecture where an LLM acts as a reasoning engine capable of: planning, tool selection, workflow execution, memory handling, decision routing, and autonomous coordination.

Traditional Chatbot vs Agentic AI

Capability	Traditional Chatbot	Agentic AI
Text generation	Yes	Yes
API calls	No	Yes
Workflow execution	No	Yes
Database access	No	Yes
Agent coordination	No	Yes
ML model calls	No	Yes
Autonomous planning	No	Yes

2. Hybrid Multi-Agent AI Architecture

Hybrid Multi-Agent AI combines LLM reasoning, predictive ML systems, workflow orchestration, enterprise tools, and databases/RAG. The LLM becomes the orchestrator while ML models act as specialised prediction engines.

Architecture Layers

Layer	Responsibility
LLM Orchestrator	Reasoning, planning, coordination, decision routing
Predictive ML	Classification, forecasting, recommendations (LR, XGBoost, DL)
Tools / APIs	Execution systems, SaaS integrations, external services
RAG + Databases	Memory, knowledge retrieval, vector search
MLOps + Monitoring	Training pipelines, drift detection, retraining automation

Handoff Types

- Agent -> Agent
- Agent -> Tool
- Agent -> ML Model
- Agent -> Human
- Agent -> Workflow Engine

Example: Router Agent -> Fraud Agent -> Refund Agent -> Human Reviewer

3. How Predictive ML Models Are Orchestrated

Predictive models are wrapped as callable tools or inference services. The agent decides: when to call the model, which features to pass, how to interpret predictions, and what action to take next.

Step-by-Step ML Orchestration Flow

Step	Phase	Description
1	Train Model	Train a predictive model (Logistic Regression / XGBoost) on labelled data.
2	Serialise Model	Save the trained model to disk using joblib/pickle.
3	Wrap as Tool	Expose the model as a callable Python function accepting feature vectors.
4	Register with Agent	Register the tool function with the agent framework.
5	Agent Invocation	The LLM orchestrator decides when to call the tool based on context.
6	Interpret Output	The agent reads prediction/probability and routes to the next step.
7	Take Action	Trigger downstream actions: notification, gameplay change, escalation.

Sample Code - Churn Prediction Tool

```
from sklearn.linear_model import LogisticRegression
import joblib

# 1. Train model
model = LogisticRegression()
model.fit(X_train, y_train)

# 2. Save model
joblib.dump(model, "churn_model.pkl")

# 3. Load model
model = joblib.load("churn_model.pkl")

# 4. Wrap as callable tool
def churn_prediction_tool(features):
    prediction  = model.predict(features)
    probability = model.predict_proba(features)
    return {
        "prediction":  int(prediction[0]),
        "probability": float(probability[0][1])
    }
```

4. Framework Deep-Dives - Step-by-Step Functionality

4.1 LangGraph

LangGraph is built on LangChain and models workflows as directed graphs with stateful nodes. It is one of the strongest frameworks for Hybrid AI systems.

Key Strengths

- Conditional routing between nodes
- Stateful orchestration (shared state dict)
- Human-in-the-loop support
- Tool orchestration
- ML workflow integration
- Cyclic graph support for iterative reasoning

Step-by-Step Functionality

Step	Phase	Description
1	Define State	Create a TypedDict/dataclass holding all shared state (features, scores, flags).
2	Create Nodes	Write Python functions for each processing step (telemetry, prediction, decision, action).
3	Build Graph	Instantiate StateGraph, add nodes, connect with edges.
4	Add Conditional Edges	Use add_conditional_edges() with a router function to branch on state values.
5	Compile Graph	Call graph.compile() to produce an executable runnable.
6	Invoke	Call graph.invoke(initial_state) to run the full workflow.
7	Inspect Output	Read the final state dict for results, scores, and action taken.

Code Example - Conditional Router

```
def predict_churn_node(state):
    features = state["customer_features"]
    result = churn_prediction_tool(features)
    state["churn_score"] = result["probability"]
    return state

def router(state):
    if state["churn_score"] > 0.8:
        return "retention_agent"
    elif state["whale_probability"] > 0.7:
        return "monetization_agent"
    return "normal_flow"

graph = StateGraph(AgentState)
graph.add_node("predict_churn", predict_churn_node)
graph.add_conditional_edges("predict_churn", router,
    {"retention_agent": "retention_agent",
     "monetization_agent": "monetization_agent",
     "normal_flow": END})
app = graph.compile()
```

4.2 Google ADK (Agent Development Kit)

Google ADK is focused on enterprise-grade multi-agent orchestration with strong governance, scalability, and integration with Google Cloud services.

Key Strengths

- Large enterprise orchestration
- AI operations & governance
- Hierarchical master/sub-agent architecture
- Native Google Cloud integration
- Built-in safety and compliance controls

Step-by-Step Functionality

Step	Phase	Description
1	Define Master Agent	Create a top-level orchestrator agent that receives user/system requests.
2	Define Sub-Agents	Create specialised agents: Telemetry, Prediction, Decision, Engagement.
3	Register Sub-Agents	Register sub-agents with the master agent as callable tools or delegates.
4	Configure Tools	Attach ML inference endpoints, APIs, and database connectors as tools.

5	Set Governance Rules	Define safety filters, approval gates, and audit logging policies.
6	Deploy on Google Cloud	Deploy agents on Vertex AI Agent Builder or Cloud Run.
7	Invoke & Monitor	Send requests to master agent; monitor via Cloud Logging & Trace.

Architecture Flow: Master Agent -> Telemetry Agent -> Prediction Agent -> Decision Agent -> Engagement Agent

---

4.3 CrewAI

CrewAI models agents like company employees with defined roles, goals, and backstories. Ideal for business workflows where human-like role assignment is needed.

Key Strengths

- Role-based agent design
- Natural language task delegation
- Sequential and parallel task execution
- Easy integration with LangChain tools
- Business-friendly abstraction

Step-by-Step Functionality

Step	Phase	Description
1	Define Agents	Create Agent objects with role, goal, backstory, and tools (e.g. Telemetry Analyst).
2	Define Tasks	Create Task objects describing what each agent must accomplish.
3	Assign Tasks	Map each task to a specific agent.
4	Create Crew	Instantiate a Crew with the list of agents and tasks.
5	Set Process	Choose Process.sequential (one after another) or Process.hierarchical.
6	Kickoff	Call crew.kickoff() to start execution; agents collaborate via shared context.
7	Collect Output	Read the final output from the last task in the pipeline.

Example Agent Roles

- Telemetry Analyst - collects and parses player session data
- Churn Specialist - interprets churn probability scores
- Monetization Strategist - identifies whale/spender opportunities
- Engagement Manager - designs and triggers retention campaigns

Flow: telemetry_agent -> churn_agent -> monetization_agent -> engagement_agent

---

4.4 AutoGen (Microsoft)

AutoGen focuses on conversational collaboration between agents. Agents communicate via structured messages, enabling iterative reasoning and peer review.

Key Strengths

- Multi-agent conversation loops
- Peer review and validation between agents

- Code execution agents
- Human proxy agent support
- Flexible termination conditions

Step-by-Step Functionality

Step	Phase	Description
1	Define Agents	Create AssistantAgent and UserProxyAgent instances with system prompts.
2	Configure LLM	Set the LLM config (model, API key, temperature) for each agent.
3	Set Termination	Define when the conversation should stop (keyword, max turns, etc.).
4	Initiate Chat	Call initiator.initiate_chat(recipient, message=...) to start the loop.
5	Agent Collaboration	Agents exchange messages: Planner -> Predictor -> Action -> Reviewer.
6	Code Execution	UserProxyAgent can execute code blocks returned by AssistantAgent.
7	Final Output	Extract the last message or structured result from the conversation.

Example Conversation Flow

```
Planner Agent:    "Player retention risk detected - churn probability high."
Prediction Agent: "Churn probability = 0.91, Whale probability = 0.45."
Action Agent:     "Triggering retention campaign: bonus coins + easier match."
Review Agent:     "Campaign execution successful. Player re-engaged."
```

4.5 AWS Bedrock Agents

AWS Bedrock Agents provide fully managed agent orchestration natively integrated with the AWS ecosystem. Best for teams already on AWS infrastructure.

Key Strengths

- Native AWS service integration
- Managed infrastructure (no server management)
- Built-in RAG via Knowledge Bases
- Action Groups for tool calling
- SageMaker endpoint integration for ML inference
- Enterprise security via IAM, GuardDuty, KMS

Step-by-Step Functionality

Step	Phase	Description
1	Choose Foundation Model	Select a model from Bedrock (Claude, Llama, Titan) as the agent LLM.
2	Create Agent	Define the agent in Bedrock console or via SDK with instructions.
3	Define Action Groups	Create Lambda functions as tools; register them as Action Groups.
4	Attach Knowledge Base	Connect an OpenSearch vector DB as a Knowledge Base for RAG.
5	Configure SageMaker	Point Action Groups to SageMaker endpoints for ML inference.
6	Set up Step Functions	Orchestrate multi-step workflows using AWS Step Functions.
7	Deploy & Invoke	Invoke via API Gateway or SDK; monitor with CloudWatch.

AWS Services Mapping

Layer	AWS Services
Foundation Models	Amazon Bedrock - Claude, Llama, Titan
Agent Orchestration	Bedrock Agents, Lambda, Step Functions

ML Training	Amazon SageMaker
Feature Store	SageMaker Feature Store
Model Registry	SageMaker Model Registry
Model Serving	SageMaker Endpoints
Data Storage	S3, DynamoDB, Aurora
Streaming	Kinesis, MSK (Kafka)
Vector Database	OpenSearch Vector Engine
Monitoring	CloudWatch, SageMaker Model Monitor
Security	IAM, GuardDuty, KMS
CI/CD	CodePipeline, CodeBuild

4.6 LlamaIndex

LlamaIndex specialises in knowledge agents and RAG orchestration. It excels at connecting LLMs to structured and unstructured data sources.

Key Strengths

- Best-in-class RAG pipelines
- Multi-document reasoning
- Query engines over structured data
- Agent + tool integration
- Flexible index types (vector, keyword, tree)

Step-by-Step Functionality

Step	Phase	Description
1	Load Documents	Use SimpleDirectoryReader or custom loaders to ingest data.
2	Build Index	Create a VectorStoreIndex or other index type from documents.
3	Create Query Engine	Instantiate a query engine from the index for retrieval.
4	Define Tools	Wrap query engines and external APIs as QueryEngineTool objects.
5	Create Agent	Instantiate an OpenAIAgent or ReActAgent with the tools.
6	Query Agent	Call agent.chat() or agent.query() with a natural language question.
7	Return Response	Agent retrieves relevant context, reasons, and returns a grounded answer.

4.7 Semantic Kernel (Microsoft)

Semantic Kernel is Microsoft's SDK for .NET, Python, and Java enterprise applications. It uses a plugin-based architecture to integrate LLMs into existing enterprise software.

Key Strengths

- .NET / C# first-class support
- Plugin-based extensibility
- Planner for automatic task decomposition
- Memory connectors (vector DBs)
- Enterprise Azure integration

Step-by-Step Functionality

Step	Phase	Description
1	Initialise Kernel	Create a Kernel instance and configure the LLM service (OpenAI, Azure OpenAI).
2	Create Plugins	Define semantic functions (prompt templates) or native functions as plugins.
3	Register Plugins	Add plugins to the kernel using kernel.add_plugin().
4	Configure Memory	Attach a vector memory store (Azure AI Search, Chroma, etc.).
5	Use Planner	Use SequentialPlanner or FunctionCallingStepwisePlanner for auto task planning.
6	Invoke Functions	Call kernel.invoke() with a function name and input variables.
7	Process Output	Read the KernelContent result and route to next business logic step.

5. Comprehensive Framework Comparison Table

Feature	LangGraph	Google ADK	CrewAI	AutoGen	Bedrock Agents	LlamaIndex	Semantic Kernel
Primary Use Case	Hybrid workflows	Enterprise orchestration	Business workflows	Collaborative agents	AWS-native systems	Knowledge/RAG agents	.NET enterprise
Key Strength	Conditional routing	Scalability & governance	Role-based agents	Agent conversations	AWS integration	RAG orchestration	Plugin workflows
Flow Model	Directed graph	Hierarchical tree	Sequential/parallel	Conversation loop	Action groups	Query pipeline	Planner-based
State Management	Built-in	Built-in	Via context	Via messages	Managed	Limited	Memory connectors
Human-in-the-Loop	Yes	Yes	Limited	Yes (UserProxy)	Yes	Limited	Limited
ML Model Integration	Native tool	Via endpoints	Via tools	Via functions	SageMaker native	Via tools	Via plugins
RAG Support	Via LangChain	Via Vertex AI	Via tools	Manual	Knowledge Bases	Best-in-class	Memory connectors
Cloud Native	Cloud-agnostic	Google Cloud	Cloud-agnostic	Cloud-agnostic	AWS only	Cloud-agnostic	Azure native
Language Support	Python	Python	Python	Python	SDK/Console	Python	.NET, Python, Java
Ease of Use	Moderate	Moderate	Easy	Easy	Moderate	Easy	Moderate
Production Readiness	High	High	Medium-High	Medium	High	Medium-High	High
Open Source	Yes	Yes	Yes	Yes	No (Managed)	Yes	Yes
Best For	Complex conditional workflows	Large enterprise governance	Role-based business tasks	Peer-review collaboration	AWS-first enterprises	Document Q&A & RAG	Microsoft/.NET enterprises

6. Framework & Infrastructure Pricing

Note: Prices are approximate as of 2024-2025 and subject to change. Always verify on official provider pricing pages.

6.1 Framework Licensing Cost

Framework	License	Base Cost	Notes
LangGraph	Open Source (MIT)	Free	LangSmith tracing: $0-$39+/mo
Google ADK	Open Source (Apache)	Free	Vertex AI usage billed separately
CrewAI	Open Source (MIT)	Free	CrewAI+ Enterprise: custom pricing
AutoGen	Open Source (MIT)	Free	Azure OpenAI usage billed separately
Bedrock Agents	Managed Service	Pay-per-use	See AWS Bedrock pricing below
LlamaIndex	Open Source (MIT)	Free	LlamaCloud: $0-custom/mo
Semantic Kernel	Open Source (MIT)	Free	Azure OpenAI usage billed separately

6.2 LLM API Pricing (per 1M tokens, approx. 2025)

Model	Input ($/1M tokens)	Output ($/1M tokens)	Notes
Claude 3.5 Sonnet (Anthropic)	$3.00	$15.00	Best balance of cost & capability
Claude 3 Haiku (Anthropic)	$0.25	$1.25	Fastest, cheapest Claude
GPT-4o (OpenAI)	$2.50	$10.00	Strong reasoning
GPT-4o mini (OpenAI)	$0.15	$0.60	Cost-effective for high volume
Gemini 1.5 Pro (Google)	$1.25	$5.00	Long context (1M tokens)
Gemini 1.5 Flash (Google)	$0.075	$0.30	Fastest Gemini
Llama 3.1 70B (self-hosted)	~$0.50-$1.00 (infra)	—	Open source, infra cost only
Amazon Titan (Bedrock)	$0.30	$0.40	AWS native, lower cost

6.3 AWS Bedrock & Infrastructure Pricing

Service	Approximate Cost	Notes
Bedrock Agent Invocation	~$0.001-$0.005 per invocation	Depends on model & steps
SageMaker ml.m5.xlarge	~$0.23/hr (on-demand)	For ML inference endpoints
SageMaker ml.g4dn.xlarge	~$0.736/hr (GPU, on-demand)	For deep learning inference
OpenSearch (t3.small)	~$0.036/hr	Vector DB for RAG
Lambda	$0.20 per 1M requests + compute	For tool/action execution
Kinesis Data Streams	$0.015/shard-hr + $0.08/1M records	For event streaming
S3 Storage	$0.023/GB/month	Data & model storage
CloudWatch Logs	$0.50/GB ingested	Monitoring & observability
Step Functions	$0.025 per 1,000 state transitions	Workflow orchestration

6.4 Google Cloud (ADK / Vertex AI) Pricing

Service	Approximate Cost	Notes
Vertex AI Agent Builder	Pay-per-use (queries)	~$0.002-$0.01 per query
Vertex AI Prediction	~$0.05-$0.50/node-hr	For ML model serving
Cloud Run	$0.00002400/vCPU-sec	For containerised agents
Firestore	$0.06/100K reads	Agent state storage
BigQuery	$5/TB queried	Analytics & feature store
Cloud Logging	$0.01/GiB after free tier	Monitoring

6.5 MLOps Stack Pricing

Tool	Approximate Cost	Notes
MLflow (self-hosted)	Free (open source)	Infra cost only
MLflow on Databricks	$0.07-$0.22/DBU	Managed MLflow
Feast Feature Store	Free (open source)	Infra cost only
Weights & Biases	Free (personal) / $50+/user/mo	Experiment tracking
SageMaker Pipelines	Included with SageMaker	Pay for compute used
Airflow (self-hosted)	Free (open source)	Infra cost only
Kubernetes (EKS)	~$0.10/hr cluster + node costs	Container orchestration

7. Real-World Use Case - Gaming Customer Retention Platform

A gaming company wants to monitor player telemetry, predict churn, personalise gameplay, automate retention campaigns, and improve monetisation.

End-to-End Workflow

Step	Phase	Description
1	Player Session Starts	Player logs in; session telemetry collection begins.
2	Telemetry Collection	Events (clicks, deaths, purchases, session length) streamed via Kinesis.
3	Feature Engineering	Raw events transformed into ML features (avg session time, spend rate, etc.).
4	ML Predictions	Three models run in parallel: Churn, Rage Quit, Whale/Spender probability.
5	Agentic Decision Engine	LLM orchestrator reads scores and routes to the appropriate agent.
6	Personalised Action	Agent triggers: bonus reward, easier match, special offer, push notification.
7	Monitoring & Feedback	Outcomes logged; models retrained on new labelled data via MLOps pipeline.

ML Models Used

- Churn Prediction - Logistic Regression / XGBoost
- Rage Quit Prediction - Gradient Boosting
- Whale/Spender Probability - XGBoost / Neural Network
- Toxicity Detection - NLP classifier

8. MLOps Architecture for Production Hybrid AI

Component	Recommended Tool	Purpose
Data Ingestion	Kafka / Kinesis	Real-time event streaming
Feature Engineering	Feast Feature Store	Consistent train/inference features
Model Training	SageMaker Pipelines / Airflow	Automated training pipelines
Experiment Tracking	MLflow / Weights & Biases	Track metrics, params, artefacts
Model Registry	SageMaker Model Registry / MLflow	Version control for models
Model Serving	SageMaker Endpoints / TorchServe	Low-latency inference APIs
CI/CD Deployment	CodePipeline / GitHub Actions	Automated model deployment
Monitoring	CloudWatch / Grafana	Latency, throughput, errors
Drift Detection	SageMaker Model Monitor	Data & concept drift alerts
Retraining Automation	Step Functions / Airflow DAGs	Trigger retraining on drift

9. Enterprise Best Practices

- 1. Use LLMs for reasoning and orchestration only - not for numerical predictions.
- 2. Use predictive ML models for numerical predictions (churn, fraud, recommendations).
- 3. Use APIs and workflows for deterministic execution steps.
- 4. Keep ML inference services separate from agent runtime for scalability.
- 5. Use feature stores for consistent training/inference feature pipelines.

- 6. Add monitoring for both model drift and LLM hallucinations.
- 7. Use event-driven systems (Kafka/Kinesis) for scalability.
- 8. Introduce human-in-the-loop gates for high-risk decisions.

10. Ultimate Mental Model

Paradigm	Core Capability
Traditional Software	Rules-based logic
Machine Learning	Predictions from data
LLMs	Reasoning and language understanding
Agentic AI	Autonomous orchestration of tools & workflows
Hybrid Multi-Agent AI	Reasoning + Predictions + Tool Orchestration + Workflow Automation